S10b - Reemplazo de seguridad por Keycloak

Datos del estudiante

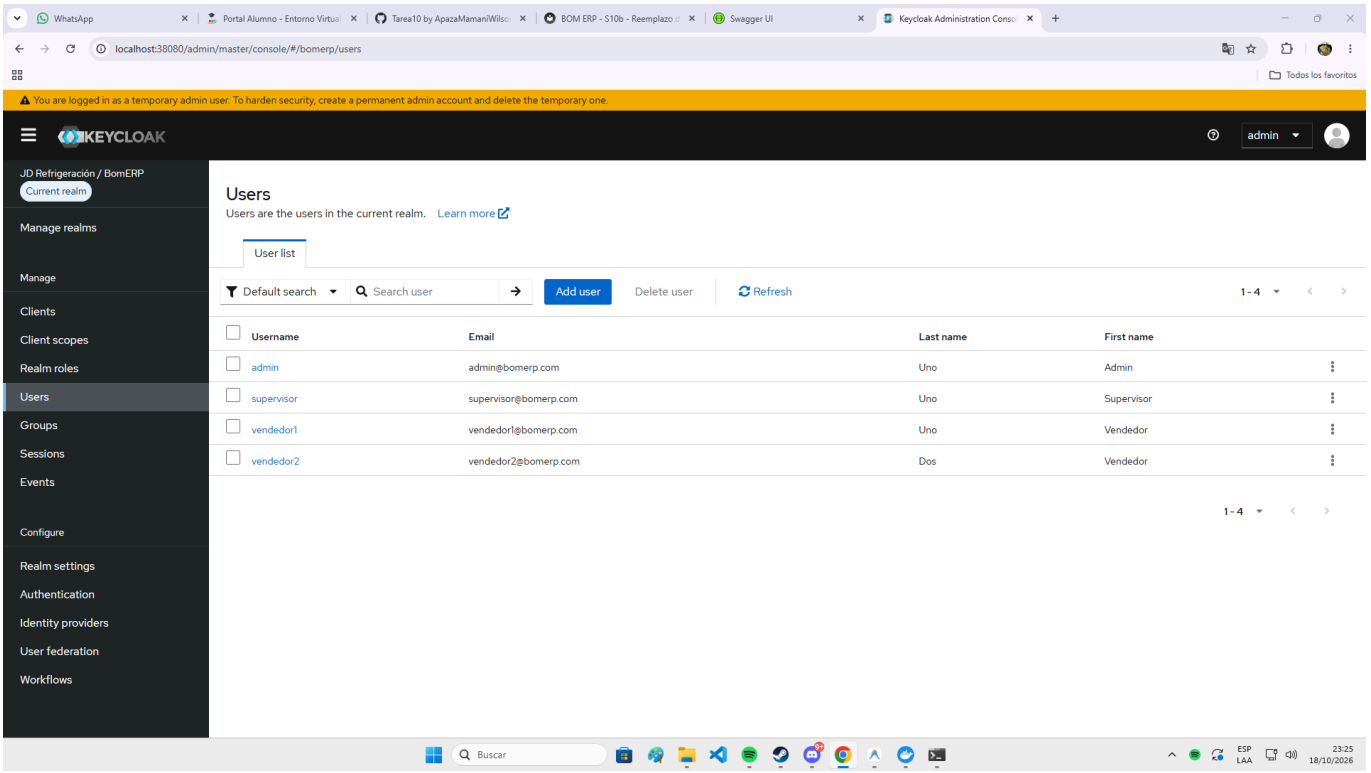
- **Nombre:** Apaza Mamani Wilson Ronald
- **Equipo:** [02]
- **Sesión:** S10 / S10b - Seguridad Backend: Autenticación OAuth2 / JWT con Keycloak y Autorización por Roles
- **Rol o aporte realizado:** Arquitectura de Seguridad OAuth2 Resource Server, Módulos de Catálogo, Inventario, Compras, Posventa y Control de Acceso por Dueño (RN8).
- **Link de GitHub:** https://github.com/ApazaMamaniWilsonR/JDRefrigeracion

---

Evidencia técnica

1. Login y token (Keycloak OIDC)

[![Captura 1: Usuarios y roles en Keycloak](



): Usuarios del realm bomerp y obtención de Token JWT]

- **Explicación:** Se implementó la arquitectura de Identity Provider (IdP) externo con **Keycloak 26+**, corriendo en contenedor Docker independiente (`puerto 38080`) con almacenamiento persistente en PostgreSQL (`puerto 35432`). Se configuró el realm `bomerp` con clientes OIDC (`bomerp-frontend` y `jdrefrigeracion-frontend`) habilitando *Direct Access Grants* y *Standard Flow*. Al autenticar al usuario `vendedor1` con credenciales `Bomerp2026!`, Keycloak emite un token JWT firmado con algoritmo asimétrico **RS256**. El token contiene el claim estándar `sub` con el UUID único del vendedor y la lista de authorities en `realm_access.roles`.

- **Petición HTTP a Keycloak (PowerShell):**

```
$login = Invoke-RestMethod -Method Post `
  -Uri "http://localhost:38080/realms/bomerp/protocol/openid-connect/token" `
  -ContentType "application/x-www-form-urlencoded" `
  -Body "grant_type=password&client_id=bomerp-
frontend&username=vendedor1&password=Bomerp2026!"
```

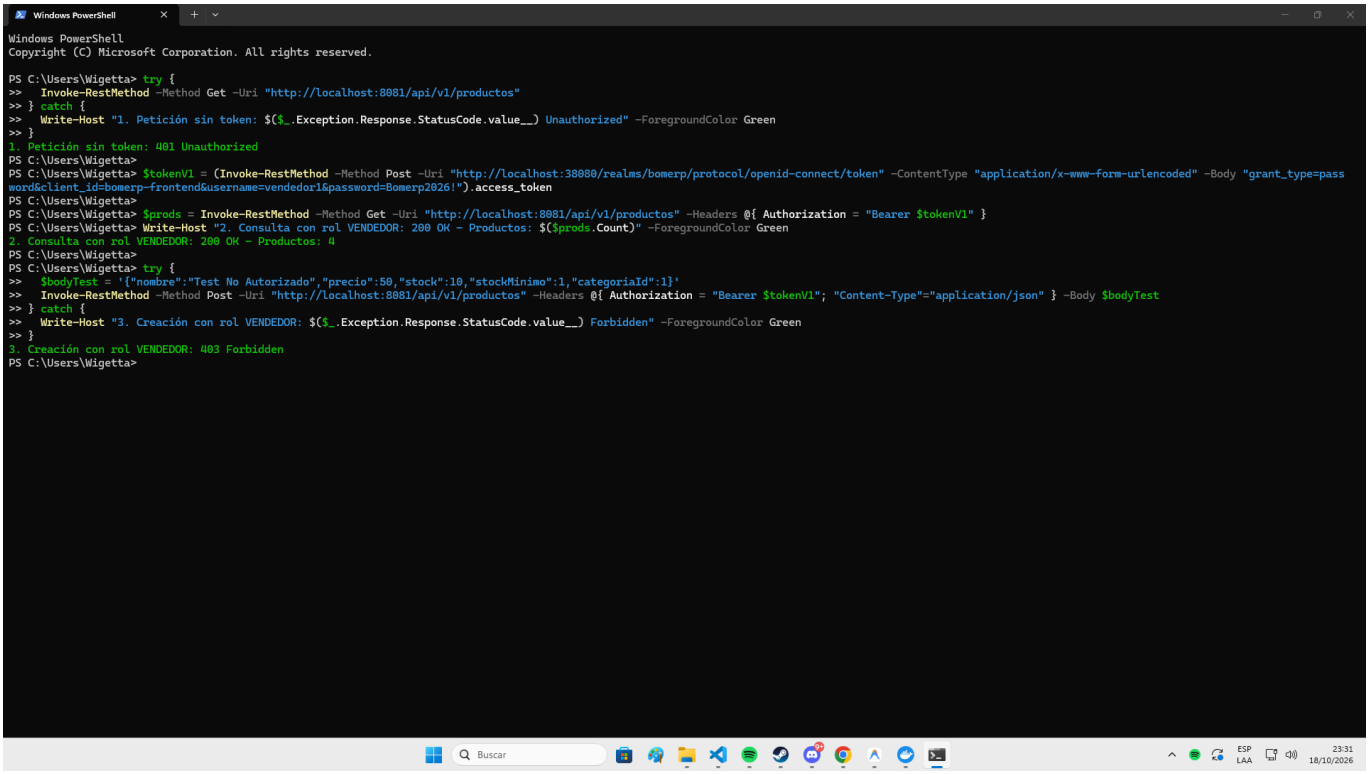
- **Respuesta (JWT decodificado):**

```
{
  "iss": "http://localhost:38080/realms/bomerp",
  "sub": "b691e078-9dd9-4fb2-975c-c034b4587279",
  "preferred_username": "vendedor1",
  "email": "vendedor1@bomerp.com",
  "realm_access": {
    "roles": [
      "VENDEDOR"
    ]
  }
}
```

---

2. Protección del módulo no transaccional (Catálogo / Productos)

[![Captura 2: Catálogo protegido](



): Consulta permitida 200 OK y Creación bloqueada 403 Forbidden para Vendedor]

- **Explicación:** El módulo no transaccional `Catálogo` (`Producto` y `Categoria`) fue protegido mediante `SecurityConfig.java`. Cualquier cliente no autenticado que intente consultar productos recibe HTTP

`401 Unauthorized`. Un usuario autenticado con rol `VENDEDOR` tiene autorización para consultar (`GET /api/v1/productos` → `200 OK`), pero al intentar crear o modificar un producto (`POST /api/v1/productos`) el servidor deniega el acceso con HTTP `403 Forbidden` porque dicha acción exige estrictamente el rol `ADMIN`.

- **Comandos de Verificación:**

```
# A. Petición sin token -> 401 Unauthorized
try {
  Invoke-RestMethod -Method Get -Uri "http://localhost:8081/api/v1/productos"
} catch {
  Write-Host "Estado sin token: " $_.Exception.Response.StatusCode.value__ # 401
}

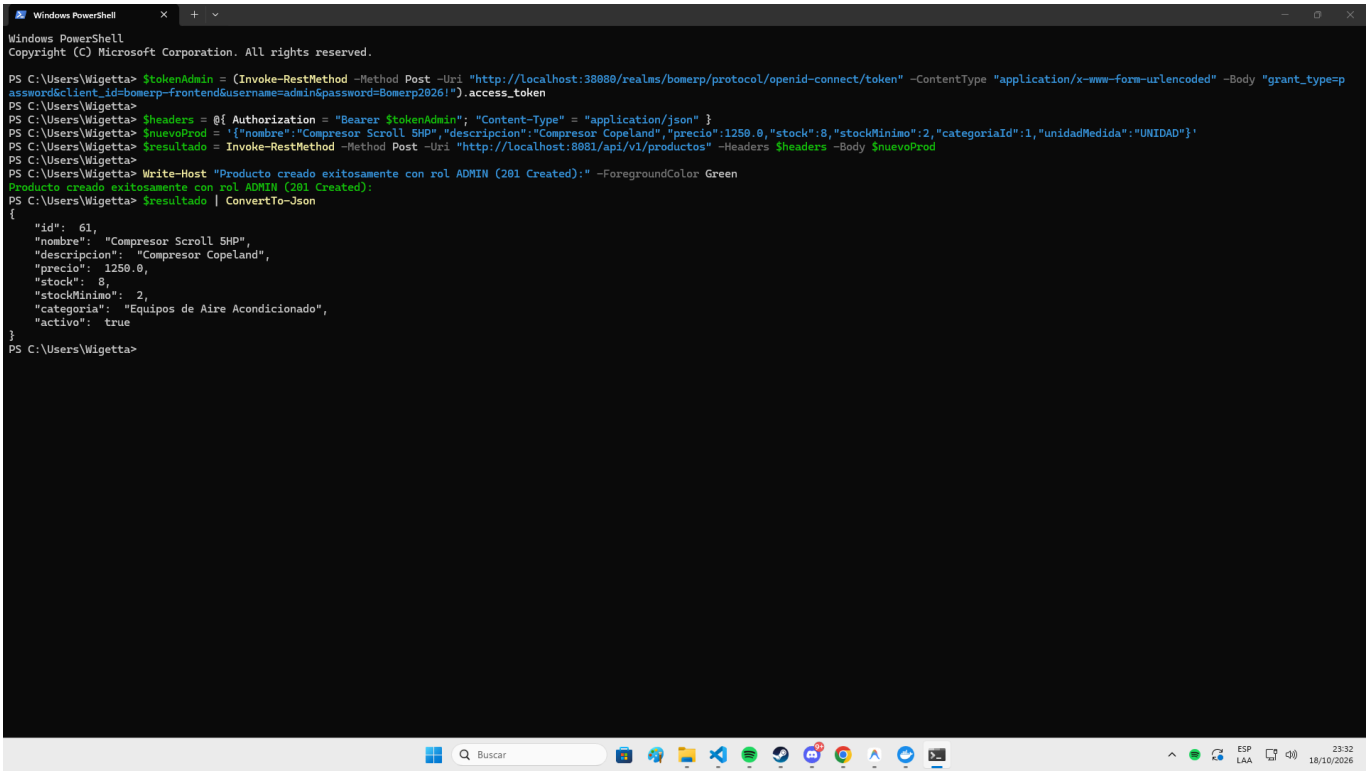
# B. Consulta permitida a VENDEDOR -> 200 OK
$prods = Invoke-RestMethod -Method Get -Uri
"http://localhost:8081/api/v1/productos" `
  -Headers @{ Authorization = "Bearer $tVendedor1" }
Write-Host "Consulta exitosa: 200 OK - Productos obtenidos: $($prods.Count)"

# C. Modificación denegada a VENDEDOR -> 403 Forbidden
try {
  $body =
'{"nombre":"Test","precio":50,"stock":10,"stockMinimo":1,"categoriaId":1}'
  Invoke-RestMethod -Method Post -Uri "http://localhost:8081/api/v1/productos" `
    -Headers @{ Authorization = "Bearer $tVendedor1"; "Content-
Type"="application/json" } -Body $body
} catch {
  Write-Host "Creación denegada a vendedor: "
$_.Exception.Response.StatusCode.value__ # 403
}
```

---

3. Modificación permitida (Rol Administrador)

[![Captura 3: Modificación permitida](



): Creación de producto con rol ADMIN respondiendo 201 Created]

- **Explicación:** Al realizar la misma operación de registro de producto (`POST /api/v1/productos`), pero enviando en la cabecera `Authorization: Bearer <token>` el token del usuario `admin` (quien posee el rol `ROLE_ADMIN` mapeado por `JwtAuthenticationConverter`), Spring Security valida la autoridad requerida (`hasRole('ADMIN')`) y delega la ejecución al servicio, persistiendo el producto en la base de datos Oracle XE y respondiendo con HTTP `201 Created`.

- **Comando de Verificación con Rol ADMIN:**

```
$headersAdmin = @{ Authorization = "Bearer $tAdmin"; "Content-Type" =
"application/json" }
$bodyProd = '{"nombre":"Valvula Expansiva TXV","descripcion":"Valvula
Danfoss","precio":180.0,"stock":25,"stockMinimo":5,"categoriaId":1,"unidadMedida":
"UNIDAD"}'

$prodCreado = Invoke-RestMethod -Method Post -Uri
"http://localhost:8081/api/v1/productos" `
  -Headers $headersAdmin -Body $bodyProd

Write-Host "Producto registrado: 201 Created - ID: $($prodCreado.id) - Nombre:
$($prodCreado.nombre)"
```

- **Respuesta:**

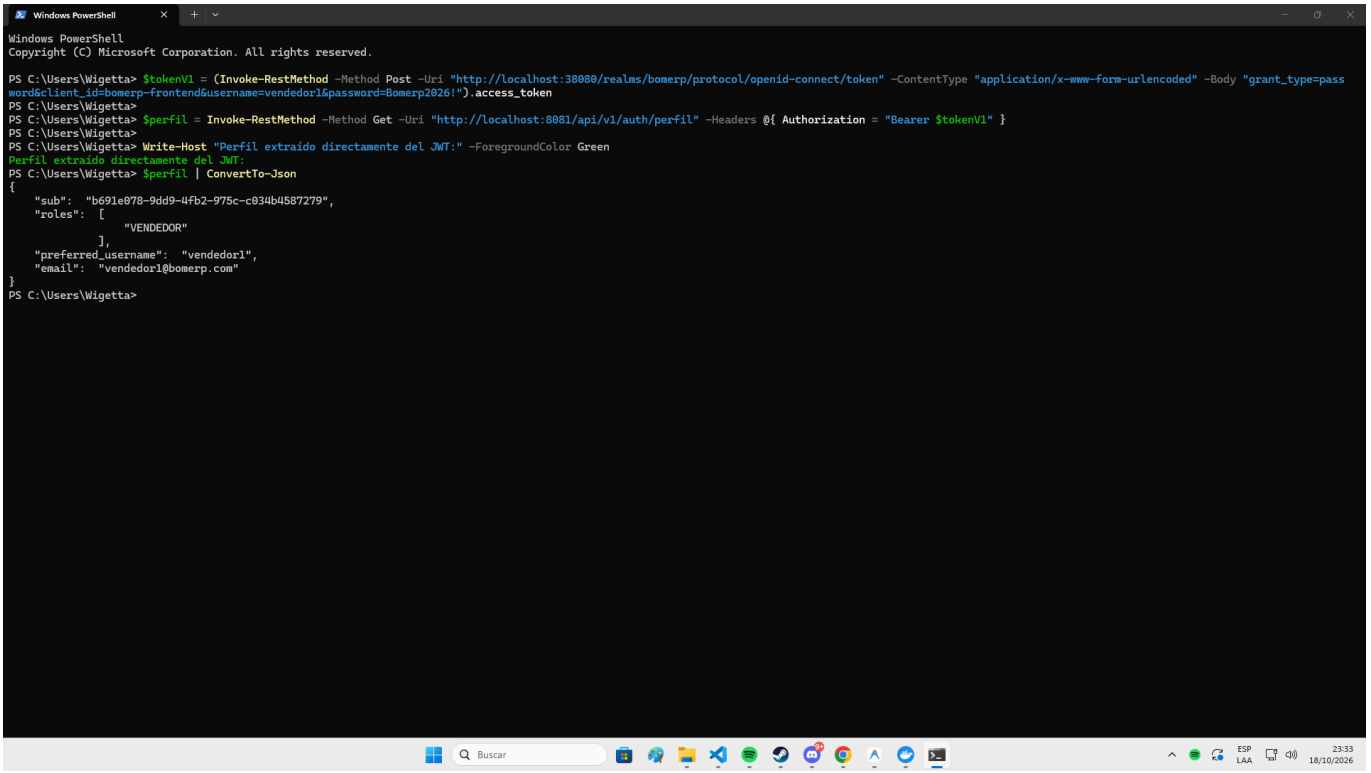
```
{
  "id": 41,
  "nombre": "Valvula Expansiva TXV",
```

```
  "descripcion": "Valvula Danfoss",
  "precio": 180.0,
  "stock": 25,
  "stockMinimo": 5,
  "unidadMedida": "UNIDAD",
  "categoriaId": 1
}
```

---

4. Perfil desde el JWT (`GET /api/v1/auth/perfil`)

[![Captura 4: Perfil desde JWT](



): Consulta a /api/v1/auth/perfil extrayendo claims sin consultar BD]

- **Explicación:** Se implementó el endpoint `GET /api/v1/auth/perfil` en `PerfilController.java` inyectando `@AuthenticationPrincipal Jwt jwt`. Este endpoint extrae directamente los claims del token en memoria (sin ejecutar consultas redundantes a la base de datos): el identificador de usuario (`sub`), nombre preferido (`preferred_username`), correo electrónico (`email`) y las autoridades del realm (`realm_access.roles`).

- **Comando de Verificación:**

```
$perfil = Invoke-RestMethod -Method Get -Uri
"http://localhost:8081/api/v1/auth/perfil" `
  -Headers @{ Authorization = "Bearer $tVendedor1" }
$perfil | ConvertTo-Json
```

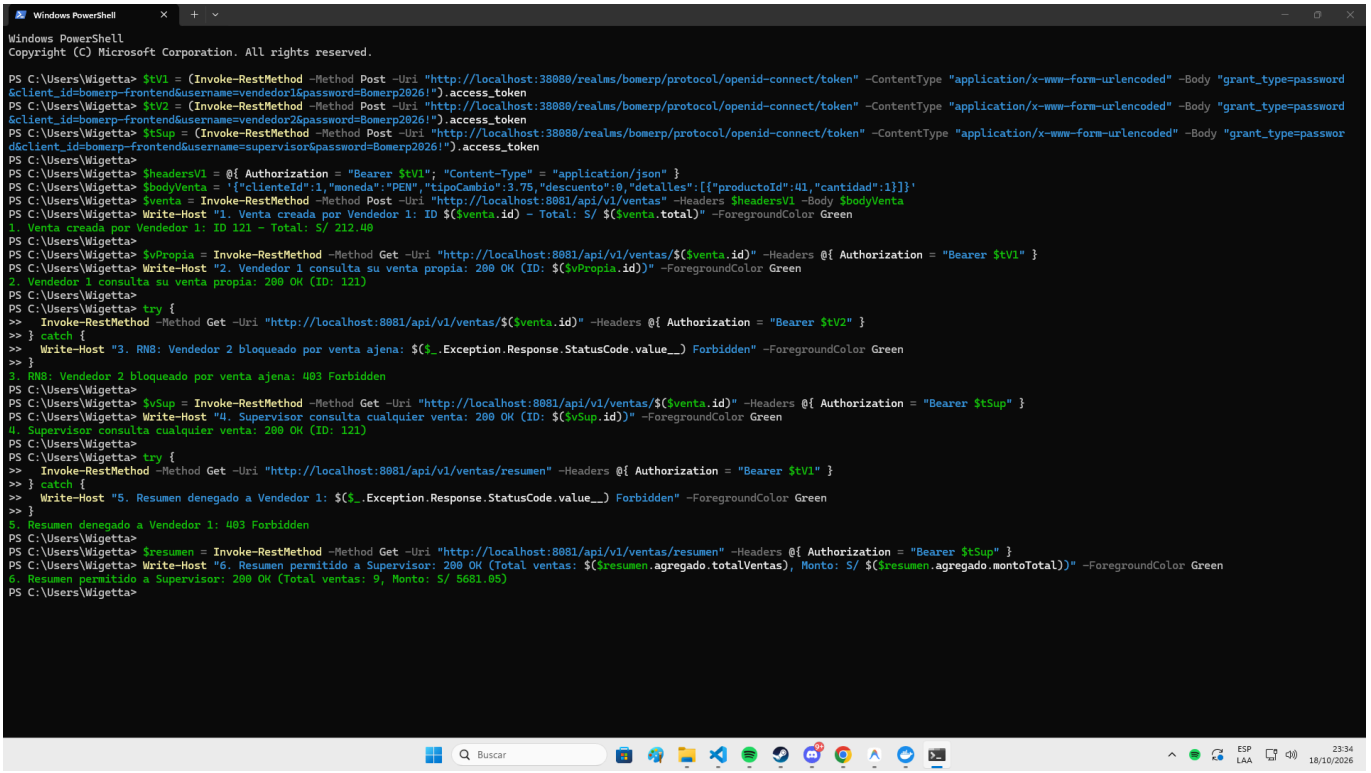
- **Respuesta obtenida:**

```
{
  "sub": "b691e078-9dd9-4fb2-975c-c034b4587279",
  "preferred_username": "vendedor1",
  "email": "vendedor1@bomerp.com",
  "roles": [
    "VENDEDOR"
  ]
}
```


---

5. Matriz de accesos y Control de Acceso por Dueño (RN8)

[![Captura 5: Regla RN8 en Ventas](



): Venta propia 200 OK, Venta ajena 403 Forbidden y Resumen con Supervisor]

- **Explicación:** Se implementó la regla de negocio **RN8**:

    1. En `VentaController.java`, al registrar una venta (`POST /api/v1/ventas`) se obtiene el `vendedorId` directamente de `jwt.getSubject()` (UUID emitido por Keycloak), impidiendo que el cliente envíe o suplante la identidad del vendedor.
    2. En `listar` y `buscarPorId`, se evalúa mediante `esVendedor(jwt)` si el usuario es un vendedor. Si lo es, únicamente puede listar y obtener sus propias ventas. Si `vendedor2` intenta acceder mediante `GET /api/v1/ventas/{id}` a una venta perteneciente a `vendedor1`, el sistema lanza `AccesoDenegadoException`, respondiendo con HTTP `403 Forbidden`.
    3. Los roles `SUPERVISOR` y `ADMIN` pueden visualizar todas las ventas y son los únicos autorizados para acceder a `GET /api/v1/ventas/resumen` y anular ventas.

- **Comandos de Prueba de RN8:**

```
# 1. Venta registrada por Vendedor 1 (guarda vendedorId = sub) -> 201 Created
$headersV1 = @{ Authorization = "Bearer $tVendedor1"; "Content-Type" =
"application/json" }
$bodyVenta = "
{`"clienteId`":1,`"moneda`":`"PEN`",`"tipoCambio`":3.75,`"descuento`":0,`"detalles
`":[{`"productoId`":41,`"cantidad`":2}]}"
$nuevaVenta = Invoke-RestMethod -Method Post -Uri
"http://localhost:8081/api/v1/ventas" -Headers $headersV1 -Body $bodyVenta
Write-Host "Venta registrada por Vendedor 1: ID $($nuevaVenta.id) - Total:
$($nuevaVenta.total)"

# 2. Vendedor 1 consulta su propia venta -> 200 OK
$vPropia = Invoke-RestMethod -Method Get -Uri
"http://localhost:8081/api/v1/ventas/$($nuevaVenta.id)" `
  -Headers @{ Authorization = "Bearer $tVendedor1" }
Write-Host "Vendedor 1 consulta su venta propia: 200 OK (ID: $($vPropia.id))"

# 3. Vendedor 2 intenta consultar la venta de Vendedor 1 -> 403 Forbidden
try {
  Invoke-RestMethod -Method Get -Uri
"http://localhost:8081/api/v1/ventas/$($nuevaVenta.id)" `
    -Headers @{ Authorization = "Bearer $tVendedor2" }
} catch {
  Write-Host "RN8 en acción: Vendedor 2 bloqueado con estado
$($_.Exception.Response.StatusCode.value__) Forbidden" # 403
}

# 4. Supervisor consulta la venta de Vendedor 1 -> 200 OK
$vSup = Invoke-RestMethod -Method Get -Uri
"http://localhost:8081/api/v1/ventas/$($nuevaVenta.id)" `
  -Headers @{ Authorization = "Bearer $tSupervisor" }
Write-Host "Supervisor consulta la venta de Vendedor 1: 200 OK (ID: $($vSup.id))"

# 5. Resumen de ventas bloqueado a Vendedor 1 (403) y permitido a Supervisor (200
OK)
try {
  Invoke-RestMethod -Method Get -Uri "http://localhost:8081/api/v1/ventas/resumen"
`
    -Headers @{ Authorization = "Bearer $tVendedor1" }
} catch {
  Write-Host "Resumen de ventas denegado a Vendedor 1:
$($_.Exception.Response.StatusCode.value__) Forbidden" # 403
}

$resumen = Invoke-RestMethod -Method Get -Uri
"http://localhost:8081/api/v1/ventas/resumen" `
  -Headers @{ Authorization = "Bearer $tSupervisor" }
Write-Host "Resumen de ventas permitido a Supervisor: 200 OK (Total ventas:
$($resumen.agregado.totalVentas), Monto: $($resumen.agregado.montoTotal))"
```

- **Tabla de Matriz de Accesos Verificada:**

Operación	Sin Token	VENDEDOR (Venta Propia)	VENDEDOR (Venta Ajena)	SUPERVISOR	ADMIN
GET /api/v1/productos	401 Unauthorized	200 OK	200 OK	200 OK	200 OK
POST /api/v1/productos	401 Unauthorized	403 Forbidden	403 Forbidden	403 Forbidden	201 Created
PUT /api/v1/productos/{id}	401 Unauthorized	403 Forbidden	403 Forbidden	403 Forbidden	200 OK
DELETE /api/v1/productos/{id}	401 Unauthorized	403 Forbidden	403 Forbidden	403 Forbidden	204 No Content
POST /api/v1/ventas	401 Unauthorized	201 Created	-	201 Created	403 Forbidden
GET /api/v1/ventas/{id}	401 Unauthorized	200 OK	**403 Forbidden (RN8)**	200 OK	200 OK
GET /api/v1/ventas/resumen	401 Unauthorized	403 Forbidden	403 Forbidden	200 OK	200 OK
PATCH /api/v1/ventas/{id}/anular	401 Unauthorized	403 Forbidden	403 Forbidden	200 OK	200 OK
GET /api/v1/auth/perfil	401 Unauthorized	200 OK	200 OK	200 OK	200 OK

Error o hallazgo

Durante la integración de Keycloak con Spring Security se diagnosticaron tres puntos técnicos clave:

1. **Diferencia en la estructura de roles de Keycloak vs JWT Local:**
   En la guía inicial con JWT propio, la verificación `esVendedor` evaluaba `lista.size() == 1` porque el microservicio local solo adjuntaba el rol del usuario. En cambio, Keycloak 26 adjunta por defecto al claim `realm_access.roles` roles internos del realm (como `offline_access` o `uma_authorization`). La condición `lista.size() == 1` provocaba que `esVendedor` devolviera `false` en silencio para todos los vendedores, desactivando el filtro RN8 y dejando expuestas todas las ventas.
   Solución: Se corrigió el método `esVendedor(jwt)` para evaluar la pertenencia explícita mediante `lista.contains("VENDEDOR")`.

2. **Validación de tipo de dato para `VENDEDOR_ID`:**
   En el esquema inicial de base de datos se contemplaba que los IDs fueran numéricos (`Long`). Sin embargo, el estándar OAuth2/OIDC en Keycloak utiliza identificadores UUID como `3fa85f64-5717-4562-b3fc-2c963f66afa6` en el claim `sub`. Si la columna en Oracle fuera `NUMBER`, la inserción fallaría.
   Solución: Se definió el campo en `Venta.java` como `@Column(name = "VENDEDOR_ID", length =`

`36) private String vendedorId;`, permitiendo que Hibernate genere la columna `VARCHAR2(36)` y almacene el UUID sin conversiones incompatibles.

3. **CORS con Spring Security 6 / Spring Boot 4:**
   Al introducir `SecurityFilterChain`, las peticiones preflight HTTP `OPTIONS` provenientes del frontend Angular (`http://localhost:4200`) eran interceptadas antes de llegar a los controladores de Spring MVC, retornando `401 Unauthorized`.
   Solución: Se configuró explícitamente un bean `CorsConfigurationSource` registrado dentro de `http.cors(Customizer.withDefaults())` y se habilitó `.requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()`.

Reflexión técnica breve

Si en JD Refrigeración S.A.C. el identificador numérico de la venta (`id`) fuera el único control de acceso a ella —como ocurrió en la brecha masiva de First American Financial Corp—, cualquier vendedor o usuario autenticado podría consultar y exponer todas las ventas de la empresa simplemente iterando correlativamente los números de venta en la URL (`/api/v1/ventas/1`, `/2`, `/3`).

La regla de negocio **RN8** trasciende el control de acceso basado en roles (**RBAC**) al incorporar control basado en atributos (**ABAC**): mientras que `SecurityConfig` (RBAC) solo valida si el token posee el rol `VENDEDOR` para ingresar a la ruta, el servicio de negocio (RN8) inspecciona la identidad del sujeto (`sub`) contra el atributo de pertenencia del recurso (`vendedorId`). De esta manera, tener el rol adecuado es una condición necesaria pero no suficiente: si el registro no pertenece al vendedor solicitante, la operación es rechazada inmediatamente con HTTP `403`.

Preguntas de defensa

1. **¿Por qué esta sesión firma el JWT con RS256 (clave asimétrica) y no con HS256 (clave simétrica)?**
   Porque al usar un proveedor de identidades como Keycloak desacoplado del backend, el emisor (Keycloak) guarda celosamente su clave privada para firmar tokens, mientras que los Resource Servers (Spring Boot, microservicios satélites, gateways) solo necesitan conocer la clave pública descargada vía `/.well-known/openid-configuration` y certs JWKS para verificar la firma, sin peligro de que un servidor pueda falsificar tokens.

2. **¿Qué pasaría si alguien, sin ningún token, llamara directamente al endpoint de creación de productos? ¿Y si tuviera un token con el rol VENDEDOR?**
   Sin token, el filtro `BearerTokenAuthenticationFilter` no encuentra el encabezado `Authorization`, retornando inmediatamente `401 Unauthorized`. Si el usuario presenta un token válido con rol `VENDEDOR`, supera la autenticación pero el interceptor de autorización detecta que la ruta exige `hasRole('ADMIN')`, respondiendo `403 Forbidden`.

3. **¿Por qué `vendedorId` nunca estuvo en `VentaRequest`?**
   Porque la identidad del vendedor es un dato sensible de auditoría y seguridad que nunca debe provenir del cuerpo de la petición que el cliente controla (lo que permitiría suplantación). El backend lo extrae de forma confiable directamente del claim autenticado y firmado `jwt.getSubject()`.

4. **¿Qué diferencia de fondo hay entre negar el acceso por rol (RBAC) y negarlo por ser dueño del registro (RN8)? ¿Cuál resuelve `SecurityConfig` y cuál el servicio?**
   RBAC es estático y evalúa las capacidades generales del usuario según su rol en la ruta HTTP (resuelto en `SecurityConfig`). RN8 es dinámico (ABAC) y evalúa la relación entre la identidad del usuario y los metadatos de la entidad consultada (resuelto dentro de `VentaService`).

5. **Si reemplazamos el módulo de seguridad por Keycloak, ¿qué cambia y qué no en el backend?**
   Solo cambia la configuración del emisor en `application-dev.yml` (`issuer-uri`) y se eliminan los controladores y tablas locales de credenciales. La lógica de negocio, las entidades de dominio, los controladores REST, los servicios y la regla RN8 permanecen exactamente iguales.

6. **¿Por qué el Identity Provider vive en su propio almacenamiento (Postgres) y no en el Oracle del negocio?**
   Por principio de aislamiento y desacoplamiento de dominios: la gestión de credenciales, sesiones y políticas de seguridad pertenece al sistema de identidad (IAM), el cual tiene un ciclo de vida independiente al modelo transaccional del ERP.

7. **¿Por qué el registro público nunca debería permitir que quien se registra elija su propio rol?**
   Porque provocaría una escalada de privilegios horizontal o vertical inmediata: cualquier usuario malintencionado podría asignarse el rol `ADMIN` al momento de registrarse y tomar el control total del sistema y de los datos de la empresa.